INTERVIEW PREPARATION HANDBOOK

Striver's A2Z DSA Handbook

The Complete Interview Preparation Guide for Product-Based Companies

Master Data Structures and Algorithms with Interview Patterns, Complexity Analysis, Java Implementations, and FAANG-Level Problem Solving

Audience	Language	Scope	Edition
Intermediate Java programmers and college students	Java-first explanations and templates	A2Z DSA topics, pattern handbooks, strategy, and appendices	2026 interview edition

Scope note: This is an original educational handbook inspired by the public Striver A2Z roadmap. It does not reproduce official solutions; it teaches the concepts, patterns, and interview reasoning needed to solve the sheet independently.

How to Use This Handbook

The fastest way to use this book is to study by pattern, then verify by solving. Each topic chapter follows the same interview loop: understand the idea, recognize the pattern, implement the template, dry-run by hand, and then practice problems in rising difficulty.

Study pass	Goal	What to do
Pass 1	Build vocabulary	Read Concept, Why it exists, Analogy, and Diagram. Do not memorize code yet.
Pass 2	Build implementation fluency	Rewrite the Java template from memory, then dry-run it on a small input.
Pass 3	Build interview judgment	Focus on tricks, pattern recognition, and how constraints guide the solution.
Pass 4	Build speed	Solve Easy to Medium first, then revisit Hard problems with a notebook of failed ideas.

Interview note: A strong candidate explains invariants and tradeoffs while coding. The code matters, but the reasoning around it is what interviewers score.

Recommended pace: finish the Foundations and Core Data Structures first, then cycle through Binary Search, Graphs, Dynamic Programming, and Advanced Structures. Use the 30/60/90-day plans near the end depending on your deadline.

Table of Contents

How to Use This Handbook 2
Table of Contents 3
DSA Pattern Recognition 6
Complexity Analysis Cheat Sheet 7
Java Collections Cheat Sheet 8
Foundations 9
Programming Basics 10
Complexity Analysis 13
Mathematics 16
Recursion 19
Hashing 22
Core Techniques 25
Sorting 26
Arrays 29
Prefix Sum 32
Difference Array 35
Kadane 38
Binary Search 41
Strings 44
Data Structures 47
Linked List 48
Stack 51
Queue 54
Monotonic Stack 57
Monotonic Queue 60
Patterns 63
Sliding Window 64
Two Pointer 67
Greedy 70
Trees 73
Binary Trees 74

BST 77
Data Structures 80
Heap 81
Trie 84
Graphs 87
Graphs 88
Topological Sorting 91
Shortest Path 94
MST 97
Disjoint Set Union 100
Advanced Graph Algorithms 103
Tarjan 106
Kosaraju 109
Dynamic Programming 112
Dynamic Programming 113
Advanced Techniques 116
Bit Manipulation 117
Backtracking 120
Advanced Data Structures 123
Segment Tree 124
Fenwick Tree 127
Advanced Trees 130
Binary Lifting 131
Euler Tour 134
Advanced Data Structures 137
Sparse Table 138
Advanced Trees 141
LCA 142
Advanced Techniques 145
Meet-in-the-Middle 146
Binary Search Pattern Handbook 148
Sliding Window Pattern Handbook 150

Graph Pattern Handbook 151
DP Pattern Handbook 152
Backtracking Pattern Handbook 153
FAANG Interview Strategy 154
OA Strategy 155
Revision Checklist 156
30-Day Revision Plan 157
60-Day Preparation Plan 158
90-Day FAANG Preparation Roadmap 159
Final 7-Day Interview Revision 160
Appendix: Big-O Cheat Sheet 161
Complexity Analysis Cheat Sheet 161
Sorting Comparison Table 162
Tree Traversal Table 163
Graph Algorithm Comparison 164
DP Pattern Comparison 165
Java Collections Comparison 166
Common Formula Sheet 167
Bit Manipulation Tricks 168
Recursion Template 169
DFS Template 170
BFS Template 171
Binary Search Template 172
Union Find Template 173
Segment Tree Template 174
Trie Template 175
Monotonic Stack Template 176
Sliding Window Template 177
Two Pointer Template 178
Fast Input Template 179
Competitive Programming Utilities 180

DSA Pattern Recognition

Pattern recognition is not guessing. It is a disciplined process of mapping problem wording, constraints, and required output to a small set of known transformations.

Signal in problem	Likely pattern	First question to ask
Contiguous subarray or substring	Sliding window, prefix sum, Kadane	Does expanding right make the condition easier to maintain?
Sorted array or monotonic answer	Binary search	Can I write a true/false check that flips once?
Pairs/triples after sorting	Two pointers	Which pointer move discards impossible pairs?
Network, grid, prerequisites	Graph traversal/topological/shortest path	Are edges weighted, directed, or cyclic?
Repeated optimal subproblems	Dynamic programming	What is the smallest state that determines the future?
Range query with updates	Fenwick or segment tree	Are updates point, range, or both?

Tip: When stuck, write the brute force recurrence or loops. The optimization usually comes from caching, sorting, indexing, or narrowing the search space.

Complexity Analysis Cheat Sheet

Notation	Meaning	Interview wording
Big-O	Upper bound on growth	The algorithm will not grow worse than this class.
Big-Theta	Tight bound	The algorithm grows exactly in this class asymptotically.
Big-Omega	Lower bound	The algorithm must take at least this much in the best guaranteed sense.
Amortized	Average over a sequence	Some operations are expensive, but the total sequence is controlled.
Recursive	Cost defined by recurrence	Count branches, work per level, and depth.
Master theorem	Shortcut for T(n)=aT(n/b)+f(n)	Compare f(n) with n^(log_b a).

- O(1): direct access or constant fixed work.
- O(log n): the search space shrinks by a factor each step.
- O(n): every element is processed a constant number of times.
- O(n log n): sorting or balanced divide-and-conquer.
- O(n^2): pair comparisons; usually too slow beyond about 10^5.
- O(2^n) and O(n!): subset/permutation search; require small n or pruning.

Warning: Never say a recursive algorithm is O(n) just because it has one parameter. Draw the recursion tree or define states.

Java Collections Cheat Sheet

Collection	Use when	Core costs	Interview warning
ArrayList	Indexed dynamic array	get O(1), add end amortized O(1)	Middle insert/delete shifts elements.
LinkedList	Deque-style operations	add/remove ends O(1)	Index access is O(n); rarely best for interviews.
ArrayDeque	Stack or queue	push/pop/offer/poll O(1)	Prefer over legacy Stack.
HashMap	Key-value lookup	average O(1)	Use getOrDefault and handle missing keys.
TreeMap	Sorted keys/range queries	O(log n)	Use floorKey/ceilingKey for neighbor queries.
HashSet	Membership	average O(1)	Custom objects need equals/hashCode.
TreeSet	Ordered set	O(log n)	Comparator must be consistent.
PriorityQueue	Heap	offer/poll O(log n)	Java PQ is min-heap by default.
Comparable	Natural ordering	sort/PQ support	Return Integer.compare(a,b), not a-b.
Comparator	Custom ordering	sort/PQ support	Avoid overflow in subtraction comparators.

```
Map<String, Integer> freq = new HashMap<>();
freq.put(word, freq.getOrDefault(word, 0) + 1);

PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
int top = stack.pop();

Arrays.sort(arr);
Collections.sort(list, Comparator.reverseOrder());
```

Foundations

This part groups the foundations topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Programming Basics

Interview lens: Programming Basics questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Programming Basics is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Programming basics exist so the interview is about problem solving, not syntax panic. Clean input, loops, conditionals, arrays, and functions let you express an algorithm precisely.

3. Real-World Analogy

Learning grammar before writing essays: you need verbs and sentence structure before you can write an argument.

4. Theory

The theory behind Programming Basics is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
input -> parse -> transform -> output
```

6. Operations

- Read input safely and convert it to typed values.
- Decompose work into functions with one responsibility.
- Use arrays, loops, and conditionals to express deterministic control flow.
- Print results exactly as required by the judge.

7. Time Complexity

Operation	Time	Space	Interview note
Input parsing	O(n)	O(1)	n tokens read once
Array traversal	O(n)	O(1)	Single pass
Nested loops	O(n*m)	O(1)	Depends on bounds

8. Java Implementation

```
import java.io.*;
import java.util.*;

public class Main {
    static class FastScanner {
        private final InputStream in = System.in;
        private final byte[] buffer = new byte[1 << 16];
        private int ptr = 0, len = 0;
        int read() throws IOException {
            if (ptr >= len) { len = in.read(buffer); ptr = 0; }
            return len <= 0 ? -1 : buffer[ptr++];
        }
        int nextInt() throws IOException {
            int c, sign = 1, val = 0;
            do { c = read(); } while (c <= ' ' && c != -1);
            if (c == '-') { sign = -1; c = read(); }
            while (c > ' ') { val = val * 10 + c - '0'; c = read(); }
            return val * sign;
        }
    }
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner();
        int n = fs.nextInt();
        long sum = 0;
        for (int i = 0; i < n; i++) sum += fs.nextInt();
        System.out.println(sum);
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Programming Basics invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Programming Basics before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Programming Basics: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.

- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Programming Basics from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Programming Basics with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Programming Basics inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Fizz Buzz	Combination Sum	Median of Two Sorted Arrays
Running Sum of 1d Array	Course Schedule	Serialize and Deserialize Binary Tree
Valid Palindrome	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Complexity Analysis

Interview lens: Complexity Analysis questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Complexity Analysis is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Complexity analysis was invented to compare algorithms without depending on a specific laptop, compiler, or input file. It predicts growth.

3. Real-World Analogy

A city traffic map: one road is fine at midnight, but you need to know what happens at rush hour.

4. Theory

The theory behind Complexity Analysis is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n)
```

6. Operations

- Count dominant loops, recursive branches, and state transitions.
- Ignore constants only after identifying the real bottleneck.
- Separate input size variables such as n, m, V, and E.
- Explain space used by auxiliary structures and recursion stack.

7. Time Complexity

Operation	Time	Space	Interview note
Single loop	O(n)	O(1)	Linear scan
Divide and conquer	O(n log n)	O(log n)	Typical merge/quick recursion
Hash lookup average	O(1)	O(n)	Worst-case can degrade

8. Java Implementation

```
class ComplexityExamples {
    static long triangularWork(int n) {
        long work = 0;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                work++;
            }
        }
        return work; // n * (n + 1) / 2 -> O(n^2)
    }

    static int binarySearchIterations(int n) {
        int count = 0;
        while (n > 1) {
            n /= 2;
            count++;
        }
        return count; // about log2(n)
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Complexity Analysis invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Complexity Analysis before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Complexity Analysis: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Complexity Analysis from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Complexity Analysis with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Complexity Analysis inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Binary Search	Combination Sum	Median of Two Sorted Arrays
Merge Sorted Array	Course Schedule	Serialize and Deserialize Binary Tree
Climbing Stairs	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Mathematics

Interview lens: Mathematics questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Mathematics is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Mathematical tools reduce brute force. GCD, primes, modular arithmetic, combinatorics, and divisibility convert a search into direct reasoning.

3. Real-World Analogy

A calculator's memory keys: small trusted formulas save repeated work.

4. Theory

The theory behind Mathematics is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
a = b*q + r
GCD(a,b) = GCD(b,r)
```

6. Operations

- Compute GCD/LCM safely with overflow awareness.
- Use modular arithmetic for large answers.
- Precompute primes or factorials when many queries repeat.
- Convert counting problems into formulas before coding.

7. Time Complexity

Operation	Time	Space	Interview note
GCD	O(log min(a,b))	O(1)	Euclid
Sieve	O(n log log n)	O(n)	Prime precomputation
Fast power	O(log n)	O(1)	Binary exponentiation

8. Java Implementation

```
class MathToolkit {
    static long gcd(long a, long b) {
        while (b != 0) {
            long t = a % b;
            a = b;
            b = t;
        }
        return Math.abs(a);
    }

    static long modPow(long a, long e, long mod) {
        long ans = 1 % mod;
        while (e > 0) {
            if ((e & 1) == 1) ans = (ans * a) % mod;
            a = (a * a) % mod;
            e >>= 1;
        }
        return ans;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Mathematics invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Mathematics before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Mathematics: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions

Level	Representative questions
Beginner	Explain Mathematics from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Mathematics with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Mathematics inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Count Primes	Combination Sum	Median of Two Sorted Arrays
Pow(x, n)	Course Schedule	Serialize and Deserialize Binary Tree
Happy Number	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Recursion

Interview lens: Recursion questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Recursion is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Recursion exists for problems whose solution contains smaller copies of itself. It makes trees, search, divide-and-conquer, and DP natural.

3. Real-World Analogy

Opening nested folders until you reach files, then returning with the answer.

4. Theory

The theory behind Recursion is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
solve(n)
 |-- solve(n-1)
 |-- combine
```

6. Operations

- Define a base case that stops the call chain.
- Define the recursive transition and what each call returns.
- Use backtracking undo steps when mutating shared state.
- Track stack depth to avoid accidental overflow.

7. Time Complexity

Operation	Time	Space	Interview note
Recursive call tree	Depends	O(depth)	Stack frames
Backtracking	O(branch^depth)	O(depth)	Search space
Memoized recursion	O(states*transitio	O(states)	Avoids repeats

Operation	Time	Space	Interview note
	n)		

8. Java Implementation

```
class RecursionTemplate {
    static void subsets(int[] a, int idx, java.util.List<Integer> path,
                        java.util.List<java.util.List<Integer>> ans) {
        if (idx == a.length) {
            ans.add(new java.util.ArrayList<>(path));
            return;
        }
        subsets(a, idx + 1, path, ans);       // do not take
        path.add(a[idx]);
        subsets(a, idx + 1, path, ans);       // take
        path.remove(path.size() - 1);         // undo
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Recursion invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Recursion before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Recursion: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions

Level	Representative questions
Beginner	Explain Recursion from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Recursion with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Recursion inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Reverse Linked List	Combination Sum	Median of Two Sorted Arrays
Subsets	Course Schedule	Serialize and Deserialize Binary Tree
Generate Parentheses	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Hashing

Interview lens: Hashing questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Hashing is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Hashing trades extra memory for fast lookup. It exists because repeated linear searches become too slow.

3. Real-World Analogy

A library index card: the title points directly to the shelf instead of scanning every book.

4. Theory

The theory behind Hashing is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
key -> hash(key) -> bucket -> value
```

6. Operations

- Insert keys with associated counts or values.
- Search for complements and previously seen states.
- Delete or decrement counts when a sliding window moves.
- Choose ordered maps when sorted traversal matters.

7. Time Complexity

Operation	Time	Space	Interview note
Insert/find/delete	O(1) avg	O(n)	HashMap/HashSet
Ordered map	O(log n)	O(n)	TreeMap
Frequency count	O(n)	O(k)	k distinct keys

8. Java Implementation

```
import java.util.*;

class FrequencyCounter {
    static Map<Integer, Integer> frequency(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);
        return freq;
    }

    static boolean hasTwoSum(int[] nums, int target) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (seen.contains(target - x)) return true;
            seen.add(x);
        }
        return false;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Hashing invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Hashing before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Hashing: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Hashing from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Hashing with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Hashing inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Two Sum	Combination Sum	Median of Two Sorted Arrays
Valid Anagram	Course Schedule	Serialize and Deserialize Binary Tree
Longest Consecutive Sequence	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Core Techniques

This part groups the core techniques topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Sorting

Interview lens: Sorting questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Sorting is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Sorting reveals structure. Many hard problems become simple when equal values group together or order enables two pointers and binary search.

3. Real-World Analogy

Arranging books alphabetically before finding duplicates.

4. Theory

The theory behind Sorting is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
[5,1,4,2] -> [1,2,4,5]
```

6. Operations

- Compare and swap or merge elements into order.
- Use stable sorting when original order matters.
- Sort custom objects with Comparator.
- Use sorting as a preprocessing step for greedy decisions.

7. Time Complexity

Operation	Time	Space	Interview note
Selection sort	O(n^2)	O(1)	Educational
Merge sort	O(n log n)	O(n)	Stable
Quick sort	O(n log n) avg	O(log n)	Worst O(n^2)

8. Java Implementation

```
class MergeSort {
    static void sort(int[] a) {
        int[] tmp = new int[a.length];
        mergeSort(a, 0, a.length - 1, tmp);
    }
    static void mergeSort(int[] a, int l, int r, int[] tmp) {
        if (l >= r) return;
        int m = l + (r - l) / 2;
        mergeSort(a, l, m, tmp);
        mergeSort(a, m + 1, r, tmp);
        int i = l, j = m + 1, k = l;
        while (i <= m && j <= r) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
        while (i <= m) tmp[k++] = a[i++];
        while (j <= r) tmp[k++] = a[j++];
        for (i = l; i <= r; i++) a[i] = tmp[i];
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Sorting invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Sorting before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Sorting: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Sorting from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Sorting with sorting, hashing, recursion, or another common pattern.

Level	Representative questions
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Sorting inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Sort Colors	Combination Sum	Median of Two Sorted Arrays
Merge Intervals	Course Schedule	Serialize and Deserialize Binary Tree
Count Inversions	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Arrays

Interview lens: Arrays questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Arrays is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Arrays are the default container for contiguous data. They exist because indexed memory gives constant-time access and compact storage.

3. Real-World Analogy

Numbered lockers in a hallway: locker 17 is reachable directly.

4. Theory

The theory behind Arrays is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
index: 0  1  2  3
value: 8  4  9  2
```

6. Operations

- Access by index.
- Traverse left-to-right or right-to-left.
- Update values in place.
- Shift values for insertion/deletion when needed.

7. Time Complexity

Operation	Time	Space	Interview note
Access	O(1)	O(1)	Index arithmetic
Search	O(n)	O(1)	Unsorted
Insert/delete middle	O(n)	O(1)	Shifting

8. Java Implementation

```
class ArrayPatterns {
    static void rotateRight(int[] a, int k) {
        int n = a.length;
        k %= n;
        reverse(a, 0, n - 1);
        reverse(a, 0, k - 1);
        reverse(a, k, n - 1);
    }
    static void reverse(int[] a, int l, int r) {
        while (l < r) {
            int t = a[l]; a[l++] = a[r]; a[r--] = t;
        }
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Arrays invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Arrays before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Arrays: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Arrays from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Arrays with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Arrays inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Move Zeroes	Combination Sum	Median of Two Sorted Arrays
Best Time to Buy and Sell Stock	Course Schedule	Serialize and Deserialize Binary Tree
Trapping Rain Water	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Prefix Sum

Interview lens: Prefix Sum questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Prefix Sum is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Prefix sums exist to answer repeated range-sum questions in constant time after one linear pass.

3. Real-World Analogy

A running bill total: to know the cost from item 4 to item 9, subtract the total before item 4.

4. Theory

The theory behind Prefix Sum is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
a:    2  1  3  4
pref: 0  2  3  6 10
sum[1..3] = pref[4]-pref[1]
```

6. Operations

- Build cumulative totals.
- Answer range queries with subtraction.
- Combine with hashing for subarray-sum counts.
- Use 2D prefix sums for grids.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to	O(1)	Depends on structure

Operation	Time	Space	Interview note
	O(log n)		

8. Java Implementation

```
class PrefixSum {
    private final long[] pref;
    PrefixSum(int[] nums) {
        pref = new long[nums.length + 1];
        for (int i = 0; i < nums.length; i++) pref[i + 1] = pref[i] + nums[i];
    }
    long rangeSum(int left, int right) { // inclusive
        return pref[right + 1] - pref[left];
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Prefix Sum invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Prefix Sum before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Prefix Sum: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Prefix Sum from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Prefix Sum with sorting, hashing, recursion, or another common pattern.

Level	Representative questions
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Prefix Sum inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Range Sum Query - Immutable	Combination Sum	Median of Two Sorted Arrays
Subarray Sum Equals K	Course Schedule	Serialize and Deserialize Binary Tree
Maximum Size Subarray Sum Equals k	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Difference Array

Interview lens: Difference Array questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Difference Array is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Difference arrays exist because many range updates can be represented by only two boundary changes.

3. Real-World Analogy

A thermostat schedule: mark when heat turns up and when it turns down, then replay the timeline.

4. Theory

The theory behind Difference Array is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
add +5 on [1,3]
diff[1]+=5, diff[4]-=5
prefix(diff) gives final values
```

6. Operations

- Mark range starts and ends.
- Reconstruct final values by prefix accumulation.
- Use for interval increments and sweep-line counts.
- Extend to 2D with rectangle corner updates.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to	O(1)	Depends on structure

Operation	Time	Space	Interview note
	O(log n)		

8. Java Implementation

```
class DifferenceArray {
    static int[] applyRangeAdds(int n, int[][] queries) {
        int[] diff = new int[n + 1];
        for (int[] q : queries) {
            int l = q[0], r = q[1], val = q[2];
            diff[l] += val;
            if (r + 1 < n) diff[r + 1] -= val;
        }
        int[] ans = new int[n];
        int cur = 0;
        for (int i = 0; i < n; i++) {
            cur += diff[i];
            ans[i] = cur;
        }
        return ans;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Difference Array invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Difference Array before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Difference Array: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Difference Array from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Difference Array with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Difference Array inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Car Pooling	Combination Sum	Median of Two Sorted Arrays
Corporate Flight Bookings	Course Schedule	Serialize and Deserialize Binary Tree
Range Addition	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Kadane

Interview lens: Kadane questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Kadane is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Kadane's algorithm exists to find the best contiguous subarray without testing all O(n^2) ranges.

3. Real-World Analogy

Running with a backpack: if the current load becomes harmful, drop it and start fresh.

4. Theory

The theory behind Kadane is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
cur = max(a[i], cur + a[i])
best = max(best, cur)
```

6. Operations

- Maintain best subarray ending at current index.
- Reset when previous sum hurts future choices.
- Track global best.
- Adapt to circular arrays and max product variants.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
class Kadane {
    static int maxSubArray(int[] nums) {
        int best = nums[0], cur = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Kadane invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Kadane before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Kadane: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Kadane from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Kadane with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Kadane inside a disguised story problem, discuss tradeoffs, and produce production-clean

Level	Representative questions
	Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Maximum Subarray	Combination Sum	Median of Two Sorted Arrays
Maximum Sum Circular Subarray	Course Schedule	Serialize and Deserialize Binary Tree
Maximum Product Subarray	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Binary Search

Interview lens: Binary Search questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Binary Search is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Binary search exists to exploit monotonic structure. Each comparison discards half the candidates.

3. Real-World Analogy

Guessing a number by asking higher or lower.

4. Theory

The theory behind Binary Search is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
false false false true true true
                ^ first true
```

6. Operations

- Search exact values in sorted arrays.
- Find lower/upper bounds.
- Search the answer space using a feasibility check.
- Avoid overflow with lo + (hi - lo) / 2.

7. Time Complexity

Operation	Time	Space	Interview note
Search sorted	O(log n)	O(1)	Invariant halves space
Lower bound	O(log n)	O(1)	First true
Answer search	O(log range * check)	O(1)	Monotonic predicate

8. Java Implementation

```
class BinarySearchPatterns {
    static int lowerBound(int[] a, int target) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] >= target) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Binary Search invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Binary Search before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Binary Search: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Binary Search from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Binary Search with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Binary Search inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Binary Search	Combination Sum	Median of Two Sorted Arrays
Search in Rotated Sorted Array	Course Schedule	Serialize and Deserialize Binary Tree
Median of Two Sorted Arrays	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Strings

Interview lens: Strings questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Strings is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

String algorithms exist because text is sequential but comparisons can be expensive. Good preprocessing avoids repeated matching.

3. Real-World Analogy

Autocomplete on a phone: the system uses structure instead of comparing every word from scratch.

4. Theory

The theory behind Strings is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
pattern: a b a b a
pi:      0 0 1 2 3
```

6. Operations

- Scan characters and maintain state.
- Normalize case or frequency when order does not matter.
- Use KMP/Z/Rabin-Karp for pattern matching.
- Use StringBuilder for repeated construction.

7. Time Complexity

Operation	Time	Space	Interview note
Index scan	O(n)	O(1)	Characters
Substring naive	O(n*m)	O(1)	May TLE
KMP	O(n+m)	O(m)	Prefix function

8. Java Implementation

```
class KMP {
    static int[] prefix(String p) {
        int[] pi = new int[p.length()];
        for (int i = 1; i < p.length(); i++) {
            int j = pi[i - 1];
            while (j > 0 && p.charAt(i) != p.charAt(j)) j = pi[j - 1];
            if (p.charAt(i) == p.charAt(j)) j++;
            pi[i] = j;
        }
        return pi;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Strings invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Strings before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Strings: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Strings from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Strings with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Strings inside a disguised story problem, discuss tradeoffs, and produce production-clean

Level	Representative questions
	Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Valid Palindrome	Combination Sum	Median of Two Sorted Arrays
Longest Palindromic Substring	Course Schedule	Serialize and Deserialize Binary Tree
Minimum Window Substring	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Data Structures

This part groups the data structures topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Linked List

Interview lens: Linked List questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Linked List is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Linked lists exist for dynamic insertion and deletion without shifting large contiguous arrays.

3. Real-World Analogy

A treasure hunt: each clue points to the next location.

4. Theory

The theory behind Linked List is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
10 -> 20 -> 30 -> NULL
```

6. Operations

- Traverse by following next pointers.
- Insert by rewiring links.
- Delete by bypassing a node.
- Reverse using prev/current/next pointers.

7. Time Complexity

Operation	Time	Space	Interview note
Access kth	O(k)	O(1)	No random access
Insert/delete known node	O(1)	O(1)	Pointer change
Reverse	O(n)	O(1)	Three pointers

8. Java Implementation

```
class LinkedListReverse {
    static class Node {
        int val;
        Node next;
        Node(int v) { val = v; }
    }
    static Node reverse(Node head) {
        Node prev = null, cur = head;
        while (cur != null) {
            Node next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Linked List invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Linked List before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Linked List: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Linked List from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Linked List with sorting, hashing, recursion, or another common pattern.

Level	Representative questions
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Linked List inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Middle of the Linked List	Combination Sum	Median of Two Sorted Arrays
Reverse Linked List	Course Schedule	Serialize and Deserialize Binary Tree
Merge k Sorted Lists	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Stack

Interview lens: Stack questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Stack is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Stacks model nested or most-recent-first work. They exist because recursion, parentheses, and undo systems need LIFO behavior.

3. Real-World Analogy

A stack of plates: the last plate placed is the first removed.

4. Theory

The theory behind Stack is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
top
30
20
10
```

6. Operations

- Push a new element.
- Pop the most recent element.
- Peek without removing.
- Use monotonic variants to answer nearest-greater/smaller questions.

7. Time Complexity

Operation	Time	Space	Interview note
Push/pop/top	O(1)	O(n)	LIFO
Expression eval	O(n)	O(n)	Operators/values
Monotonic stack	O(n)	O(n)	Each item enters/exits once

8. Java Implementation

```
import java.util.*;

class MinStack {
    private final Deque<Integer> st = new ArrayDeque<>();
    private final Deque<Integer> min = new ArrayDeque<>();
    void push(int x) {
        st.push(x);
        if (min.isEmpty() || x <= min.peek()) min.push(x);
    }
    int pop() {
        int x = st.pop();
        if (x == min.peek()) min.pop();
        return x;
    }
    int getMin() { return min.peek(); }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Stack invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Stack before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Stack: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Stack from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Stack with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Stack inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Valid Parentheses	Combination Sum	Median of Two Sorted Arrays
Min Stack	Course Schedule	Serialize and Deserialize Binary Tree
Largest Rectangle in Histogram	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Queue

> **Interview lens:** Queue questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Queue is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Queues model first-come-first-served processing. They exist for scheduling, BFS, and streaming buffers.

3. Real-World Analogy

People standing in a ticket line.

4. Theory

The theory behind Queue is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
Front -> 10 20 30 <- Rear
```

6. Operations

- Enqueue at the rear.
- Dequeue from the front.
- Peek the front element.
- Use queue layers for BFS shortest steps in unweighted graphs.

7. Time Complexity

Operation	Time	Space	Interview note
Enqueue/dequeue	O(1)	O(n)	FIFO
BFS	O(V+E)	O(V)	Layer order
Circular buffer	O(1)	O(n)	Fixed capacity

8. Java Implementation

```
class CircularQueue {
    private final int[] a;
    private int head = 0, tail = 0, size = 0;
    CircularQueue(int capacity) { a = new int[capacity]; }
    boolean offer(int x) {
        if (size == a.length) return false;
        a[tail] = x;
        tail = (tail + 1) % a.length;
        size++;
        return true;
    }
    int poll() {
        if (size == 0) throw new java.util.NoSuchElementException();
        int x = a[head];
        head = (head + 1) % a.length;
        size--;
        return x;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Queue invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Queue before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Queue: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Queue from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Queue with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Queue inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Implement Queue using Stacks	Combination Sum	Median of Two Sorted Arrays
Rotting Oranges	Course Schedule	Serialize and Deserialize Binary Tree
Number of Islands	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Monotonic Stack

Interview lens: Monotonic Stack questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Monotonic Stack is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

A monotonic stack exists to remember only candidates that can still become the answer.

3. Real-World Analogy

A line of taller buildings blocking shorter buildings behind them.

4. Theory

The theory behind Monotonic Stack is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
incoming x pops smaller elements
stack remains decreasing
```

6. Operations

- Push indexes, not just values, when distances are needed.
- Pop while the invariant is violated.
- Answer for popped elements immediately.
- Handle equal values deliberately.

7. Time Complexity

Operation	Time	Space	Interview note
Push/pop/top	O(1)	O(n)	LIFO
Expression eval	O(n)	O(n)	Operators/values
Monotonic stack	O(n)	O(n)	Each item enters/exits once

8. Java Implementation

```
import java.util.*;

class NextGreaterElement {
    static int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        Deque<Integer> st = new ArrayDeque<>(); // indexes with decreasing values
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && nums[i] > nums[st.peek()]) ans[st.pop()] = nums[i];
            st.push(i);
        }
        return ans;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Monotonic Stack invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Monotonic Stack before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Monotonic Stack: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Monotonic Stack from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Monotonic Stack with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Monotonic Stack inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Next Greater Element I	Combination Sum	Median of Two Sorted Arrays
Daily Temperatures	Course Schedule	Serialize and Deserialize Binary Tree
Largest Rectangle in Histogram	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Monotonic Queue

Interview lens: Monotonic Queue questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Monotonic Queue is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

A monotonic queue exists to answer sliding-window min/max queries in linear time.

3. Real-World Analogy

A shortlist of the strongest candidates; anyone worse and older gets removed.

4. Theory

The theory behind Monotonic Queue is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
window [l..r]
front = best value
back = newest candidate
```

6. Operations

- Remove expired indexes from the front.
- Pop worse candidates from the back.
- Read best value at the front.
- Push current index after cleanup.

7. Time Complexity

Operation	Time	Space	Interview note
Enqueue/dequeue	O(1)	O(n)	FIFO
BFS	O(V+E)	O(V)	Layer order
Circular buffer	O(1)	O(n)	Fixed capacity

8. Java Implementation

```
import java.util.*;

class SlidingWindowMaximum {
    static int[] maxSlidingWindow(int[] nums, int k) {
        int[] ans = new int[nums.length - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        for (int i = 0; i < nums.length; i++) {
            while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
            while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast();
            dq.offerLast(i);
            if (i >= k - 1) ans[i - k + 1] = nums[dq.peekFirst()];
        }
        return ans;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Monotonic Queue invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Monotonic Queue before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Monotonic Queue: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Monotonic Queue from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Monotonic Queue with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Monotonic Queue inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Sliding Window Maximum	Combination Sum	Median of Two Sorted Arrays
Shortest Subarray with Sum at Least K	Course Schedule	Serialize and Deserialize Binary Tree
Constrained Subsequence Sum	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Patterns

This part groups the patterns topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Sliding Window

Interview lens: Sliding Window questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Sliding Window is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Sliding window exists for contiguous subarray/substring problems where expanding and shrinking a range preserves enough information.

3. Real-World Analogy

Moving a camera frame across a long table.

4. Theory

The theory behind Sliding Window is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
left        right
  |-----------|
```

6. Operations

- Expand right to include new data.
- Shrink left while the window violates a condition.
- Maintain counts, sum, or distinct elements.
- Convert exactly-K counts with atMost(K)-atMost(K-1).

7. Time Complexity

Operation	Time	Space	Interview note
Fixed window	O(n)	O(1)	Add right/remove left
Variable window	O(n)	O(k)	Each pointer moves once
Exactly K	O(n)	O(k)	atMost(K)-atMost(K-1)

8. Java Implementation

```
import java.util.*;

class LongestAtMostKDistinct {
    static int longest(String s, int k) {
        Map<Character, Integer> count = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            count.put(s.charAt(right), count.getOrDefault(s.charAt(right), 0) + 1);
            while (count.size() > k) {
                char c = s.charAt(left++);
                count.put(c, count.get(c) - 1);
                if (count.get(c) == 0) count.remove(c);
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Sliding Window invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Sliding Window before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Sliding Window: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Sliding Window from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Sliding Window with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Sliding Window inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Maximum Average Subarray I	Combination Sum	Median of Two Sorted Arrays
Longest Substring Without Repeating Characters	Course Schedule	Serialize and Deserialize Binary Tree
Minimum Window Substring	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Two Pointer

Interview lens: Two Pointer questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Two Pointer is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Two pointers exist when one pass with coordinated indexes can replace nested loops.

3. Real-World Analogy

Two people walking from opposite ends of a bookshelf toward the right pair.

4. Theory

The theory behind Two Pointer is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
l -> 1 3 4 8 10 <- r
```

6. Operations

- Move the pointer whose side cannot produce a better answer.
- Use fast/slow pointers for linked-list cycles and middle nodes.
- Merge two sorted sequences.
- Skip duplicates carefully in combination problems.

7. Time Complexity

Operation	Time	Space	Interview note
Opposite ends	O(n)	O(1)	Sorted array
Fast/slow	O(n)	O(1)	Cycle/middle
Merge	O(n+m)	O(1)	Two sorted streams

8. Java Implementation

```
class TwoPointerPairSum {
    static boolean hasPair(int[] sorted, int target) {
        int l = 0, r = sorted.length - 1;
        while (l < r) {
            int sum = sorted[l] + sorted[r];
            if (sum == target) return true;
            if (sum < target) l++;
            else r--;
        }
        return false;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Two Pointer invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Two Pointer before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Two Pointer: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Two Pointer from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Two Pointer with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Two Pointer inside a disguised story problem, discuss tradeoffs, and produce production-

Level	Representative questions
	clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Two Sum II	Combination Sum	Median of Two Sorted Arrays
Container With Most Water	Course Schedule	Serialize and Deserialize Binary Tree
3Sum	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Greedy

Interview lens: Greedy questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Greedy is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Greedy algorithms exist when a locally best choice can be proven not to harm the global optimum.

3. Real-World Analogy

Choosing the earliest-finishing meeting so the room frees up soonest.

4. Theory

The theory behind Greedy is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
sort -> choose valid best -> repeat
```

6. Operations

- Find the exchange argument or invariant.
- Sort by the criterion that makes the choice safe.
- Use heap when the current best changes dynamically.
- Reject greedy if a local choice can trap future options.

7. Time Complexity

Operation	Time	Space	Interview note
Sort + choose	O(n log n)	O(1)	Ordering dominates
Heap greedy	O(n log n)	O(n)	Repeated best choice
Interval greedy	O(n log n)	O(1)	Earliest finish

8. Java Implementation

```
import java.util.*;

class IntervalScheduling {
    static int maxNonOverlapping(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt(a -> a[1]));
        int count = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] in : intervals) {
            if (in[0] >= lastEnd) {
                count++;
                lastEnd = in[1];
            }
        }
        return count;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Greedy invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Greedy before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Greedy: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Greedy from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Greedy with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Greedy inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Assign Cookies	Combination Sum	Median of Two Sorted Arrays
Jump Game	Course Schedule	Serialize and Deserialize Binary Tree
Non-overlapping Intervals	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Trees

This part groups the trees topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Binary Trees

Interview lens: Binary Trees questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Binary Trees is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Trees model hierarchy. Binary trees appear in parsing, search structures, decision trees, and recursive decomposition.

3. Real-World Analogy

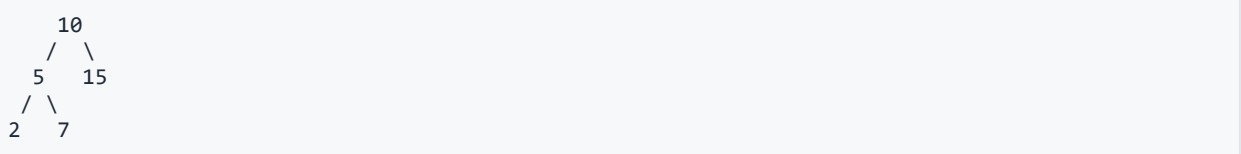
An org chart where each manager has up to two direct branches.

4. Theory

The theory behind Binary Trees is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram



6. Operations

- Traverse preorder, inorder, postorder, and level order.
- Compute height, diameter, balance, and views.
- Build trees from traversal arrays.
- Use recursion to return facts from children to parent.

7. Time Complexity

Operation	Time	Space	Interview note
DFS traversal	O(n)	O(h)	Recursion stack
BFS traversal	O(n)	O(width)	Queue

Operation	Time	Space	Interview note
Height/balance	O(n)	O(h)	Postorder

8. Java Implementation

```
import java.util.*;

class TreeTraversal {
    static class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }
    static List<Integer> inorder(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            ans.add(cur.val);
            cur = cur.right;
        }
        return ans;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Binary Trees invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Binary Trees before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Binary Trees: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Binary Trees from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Binary Trees with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Binary Trees inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Maximum Depth of Binary Tree	Combination Sum	Median of Two Sorted Arrays
Binary Tree Level Order Traversal	Course Schedule	Serialize and Deserialize Binary Tree
Serialize and Deserialize Binary Tree	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

BST

Interview lens: BST questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

BST is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

A BST exists to keep values ordered so search, insertion, and range queries follow one branch instead of scanning everything.

3. Real-World Analogy

A dictionary split by alphabet ranges.

4. Theory

The theory behind BST is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
left values < node < right values
```

6. Operations

- Search by comparing with node value.
- Insert at the first null branch.
- Delete with leaf/one-child/two-child cases.
- Use inorder traversal for sorted output.

7. Time Complexity

Operation	Time	Space	Interview note
Search/insert/delete	O(h)	O(h)	h can be n
Balanced search	O(log n)	O(log n)	If height controlled
Inorder	O(n)	O(h)	Sorted order

8. Java Implementation

```
class ValidateBST {
    static class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }
    static boolean isValid(TreeNode root) {
        return dfs(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
    static boolean dfs(TreeNode node, long low, long high) {
        if (node == null) return true;
        if (node.val <= low || node.val >= high) return false;
        return dfs(node.left, low, node.val) && dfs(node.right, node.val, high);
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the BST invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for BST before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies BST: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain BST from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine BST with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize BST inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Search in a Binary Search Tree	Combination Sum	Median of Two Sorted Arrays
Validate Binary Search Tree	Course Schedule	Serialize and Deserialize Binary Tree
Kth Smallest Element in a BST	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Data Structures

This part groups the data structures topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Heap

Interview lens: Heap questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Heap is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Heaps exist for repeated access to the smallest or largest current item.

3. Real-World Analogy

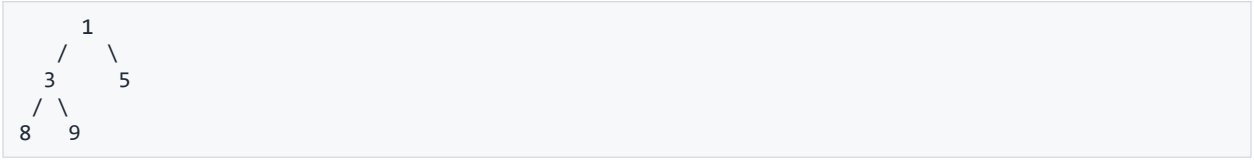
A hospital emergency queue where highest priority is treated first.

4. Theory

The theory behind Heap is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram



6. Operations

- Offer an element and bubble it upward.
- Poll root and heapify downward.
- Peek the priority element.
- Use two heaps for medians or scheduling.

7. Time Complexity

Operation	Time	Space	Interview note
Push/pop	O(log n)	O(n)	PriorityQueue
Peek	O(1)	O(n)	Best element
Build heap	O(n)	O(n)	Heapify

8. Java Implementation

```
import java.util.*;

class TopKFrequent {
    static List<Integer> topK(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);
        PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.comparingInt(freq::get));
        for (int x : freq.keySet()) {
            pq.offer(x);
            if (pq.size() > k) pq.poll();
        }
        return new ArrayList<>(pq);
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Heap invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Heap before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Heap: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Heap from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Heap with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Heap inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Kth Largest Element in an Array	Combination Sum	Median of Two Sorted Arrays
Top K Frequent Elements	Course Schedule	Serialize and Deserialize Binary Tree
Find Median from Data Stream	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Trie

Interview lens: Trie questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Trie is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Tries exist to share prefixes between words and answer prefix queries efficiently.

3. Real-World Analogy

Mobile keyboard autocomplete.

4. Theory

The theory behind Trie is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
root
 |-- c -- a -- t
 |        \-- r
```

6. Operations

- Insert characters node by node.
- Search full words with end markers.
- Search prefixes without requiring end markers.
- Use bitwise tries for maximum XOR.

7. Time Complexity

Operation	Time	Space	Interview note
Insert/search	O(L)	O(nodes)	L word length
Prefix query	O(L)	O(nodes)	Autocomplete
Bitwise trie	O(bits)	O(n*bits)	XOR problems

8. Java Implementation

```
class Trie {
    static class Node {
        Node[] next = new Node[26];
        boolean end;
    }
    private final Node root = new Node();
    void insert(String word) {
        Node cur = root;
        for (char ch : word.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new Node();
            cur = cur.next[i];
        }
        cur.end = true;
    }
    boolean startsWith(String prefix) {
        Node cur = root;
        for (char ch : prefix.toCharArray()) {
            cur = cur.next[ch - 'a'];
            if (cur == null) return false;
        }
        return true;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Trie invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Trie before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Trie: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Trie from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Trie with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Trie inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Implement Trie	Combination Sum	Median of Two Sorted Arrays
Word Search II	Course Schedule	Serialize and Deserialize Binary Tree
Maximum XOR of Two Numbers in an Array	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Graphs

This part groups the graphs topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Graphs

Interview lens: Graphs questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Graphs is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Graphs exist because many relationships are networks rather than lines or hierarchies.

3. Real-World Analogy

Google Maps: cities are nodes, roads are edges.

4. Theory

The theory behind Graphs is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
0 -- 1
|    |
2 -- 3
```

6. Operations

- Represent with adjacency list, matrix, or grid offsets.
- Traverse with BFS or DFS.
- Detect connected components, cycles, and bipartition.
- Respect directed vs undirected edge semantics.

7. Time Complexity

Operation	Time	Space	Interview note
DFS/BFS	O(V+E)	O(V)	Adjacency list
Matrix scan	O(V^2)	O(V^2)	Dense graph
Grid graph	O(R*C)	O(R*C)	Cells as vertices

8. Java Implementation

```
import java.util.*;

class GraphTraversal {
    static List<Integer> bfs(List<List<Integer>> g, int start) {
        boolean[] seen = new boolean[g.size()];
        List<Integer> order = new ArrayList<>();
        Queue<Integer> q = new ArrayDeque<>();
        seen[start] = true;
        q.offer(start);
        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g.get(u)) if (!seen[v]) {
                seen[v] = true;
                q.offer(v);
            }
        }
        return order;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Graphs invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Graphs before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Graphs: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions

Level	Representative questions
Beginner	Explain Graphs from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Graphs with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Graphs inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Number of Islands	Combination Sum	Median of Two Sorted Arrays
Clone Graph	Course Schedule	Serialize and Deserialize Binary Tree
Course Schedule	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Topological Sorting

Interview lens: Topological Sorting questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Topological Sorting is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Topological sorting exists to order tasks that have prerequisites.

3. Real-World Analogy

Taking courses only after completing prerequisites.

4. Theory

The theory behind Topological Sorting is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
A -> C
B -> C
order: A, B, C
```

6. Operations

- Compute indegrees.
- Start with zero-indegree nodes.
- Remove edges as tasks complete.
- If processed count is smaller than V, a cycle exists.

7. Time Complexity

Operation	Time	Space	Interview note
Selection sort	O(n^2)	O(1)	Educational
Merge sort	O(n log n)	O(n)	Stable
Quick sort	O(n log n) avg	O(log n)	Worst O(n^2)

8. Java Implementation

```
import java.util.*;

class KahnTopo {
    static List<Integer> topo(int n, int[][] edges) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        int[] indeg = new int[n];
        for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }
        Queue<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
        List<Integer> ans = new ArrayList<>();
        while (!q.isEmpty()) {
            int u = q.poll();
            ans.add(u);
            for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
        }
        return ans.size() == n ? ans : List.of(); // empty means cycle
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Topological Sorting invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Topological Sorting before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Topological Sorting: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Topological Sorting from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Topological Sorting with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Topological Sorting inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Course Schedule	Combination Sum	Median of Two Sorted Arrays
Course Schedule II	Course Schedule	Serialize and Deserialize Binary Tree
Alien Dictionary	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Shortest Path

Interview lens: Shortest Path questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Shortest Path is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Shortest path algorithms exist to minimize cost through a network under different edge-weight rules.

3. Real-World Analogy

Navigation choosing the fastest route, not simply the fewest turns.

4. Theory

The theory behind Shortest Path is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
unweighted -> BFS
nonnegative -> Dijkstra
negative edges -> Bellman-Ford
```

6. Operations

- Choose algorithm by edge weights.
- Relax edges when a better distance appears.
- Use priority queues for Dijkstra.
- Detect negative cycles when required.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to	O(1)	Depends on structure

Operation	Time	Space	Interview note
	O(log n)		

8. Java Implementation

```
import java.util.*;

class Dijkstra {
    static long[] shortest(int n, List<int[]>[] g, int src) {
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE / 4);
        dist[src] = 0;
        PriorityQueue<long[]> pq = new PriorityQueue<>(Comparator.comparingLong(a -> a[1]));
        pq.offer(new long[]{src, 0});
        while (!pq.isEmpty()) {
            long[] cur = pq.poll();
            int u = (int) cur[0];
            if (cur[1] != dist[u]) continue;
            for (int[] e : g[u]) {
                int v = e[0], w = e[1];
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    pq.offer(new long[]{v, dist[v]});
                }
            }
        }
        return dist;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Shortest Path invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Shortest Path before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Shortest Path: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.

- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Shortest Path from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Shortest Path with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Shortest Path inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Network Delay Time	Combination Sum	Median of Two Sorted Arrays
Cheapest Flights Within K Stops	Course Schedule	Serialize and Deserialize Binary Tree
Path With Minimum Effort	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

MST

Interview lens: MST questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

MST is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Minimum spanning trees exist to connect all nodes with minimum total edge cost and no cycles.

3. Real-World Analogy

Laying cable between offices as cheaply as possible while keeping everyone connected.

4. Theory

The theory behind MST is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
sort edges by weight -> add if it connects components
```

6. Operations

- Sort edges for Kruskal.
- Use DSU to avoid cycles.
- Use Prim with a heap from any start.
- Verify graph connectivity if all nodes must be connected.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
import java.util.*;

class KruskalMST {
    static class DSU {
        int[] p, sz;
        DSU(int n) { p = new int[n]; sz = new int[n]; for (int i = 0; i < n; i++) { p[i] = i;
sz[i] = 1; } }
        int find(int x) { return p[x] == x ? x : (p[x] = find(p[x])); }
        boolean union(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (sz[a] < sz[b]) { int t = a; a = b; b = t; }
            p[b] = a; sz[a] += sz[b]; return true;
        }
    }
    static int mstCost(int n, int[][] edges) {
        Arrays.sort(edges, Comparator.comparingInt(e -> e[2]));
        DSU dsu = new DSU(n);
        int cost = 0;
        for (int[] e : edges) if (dsu.union(e[0], e[1])) cost += e[2];
        return cost;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the MST invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for MST before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies MST: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain MST from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine MST with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize MST inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Min Cost to Connect All Points	Combination Sum	Median of Two Sorted Arrays
Connecting Cities With Minimum Cost	Course Schedule	Serialize and Deserialize Binary Tree
Optimize Water Distribution in a Village	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Disjoint Set Union

Interview lens: Disjoint Set Union questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Disjoint Set Union is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

DSU exists to maintain dynamic connectivity between components with near-constant operations.

3. Real-World Analogy

Merging friend groups and asking whether two people are already in the same group.

4. Theory

The theory behind Disjoint Set Union is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
parent[x] -> representative
```

6. Operations

- Find the representative of a node.
- Compress paths during find.
- Union smaller tree into larger tree.
- Use for cycles, components, and Kruskal MST.

7. Time Complexity

Operation	Time	Space	Interview note
Find	Alpha(n)	O(n)	Path compression
Union	Alpha(n)	O(n)	By size/rank
Kruskal	O(E log E)	O(V)	Sort edges

8. Java Implementation

```
class UnionFind {
    int[] parent, size;
    UnionFind(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) { int t = a; a = b; b = t; }
        parent[b] = a; size[a] += size[b];
        return true;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Disjoint Set Union invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Disjoint Set Union before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Disjoint Set Union: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Disjoint Set Union from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Disjoint Set Union with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Disjoint Set Union inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Number of Connected Components in an Undirected Graph	Combination Sum	Median of Two Sorted Arrays
Redundant Connection	Course Schedule	Serialize and Deserialize Binary Tree
Accounts Merge	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Graph Algorithms

Interview lens: Advanced Graph Algorithms questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Advanced Graph Algorithms is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Advanced graph algorithms exist for structure beyond reachability: cuts, components, parity, flows, and dependency condensation.

3. Real-World Analogy

Urban planning: not just whether roads exist, but which bridges are critical and which districts are inseparable.

4. Theory

The theory behind Advanced Graph Algorithms is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
graph -> structural property -> compressed answer
```

6. Operations

- Classify graph type and constraints.
- Track discovery times or colors.
- Compress components when cycles obscure DAG structure.
- Use the weakest algorithm that proves the property needed.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries

Operation	Time	Space	Interview note
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
import java.util.*;

class BipartiteCheck {
    static boolean isBipartite(List<List<Integer>> g) {
        int[] color = new int[g.size()];
        Arrays.fill(color, -1);
        for (int s = 0; s < g.size(); s++) if (color[s] == -1) {
            Queue<Integer> q = new ArrayDeque<>();
            color[s] = 0; q.offer(s);
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : g.get(u)) {
                    if (color[v] == -1) { color[v] = color[u] ^ 1; q.offer(v); }
                    else if (color[v] == color[u]) return false;
                }
            }
        }
        return true;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Advanced Graph Algorithms invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Advanced Graph Algorithms before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Advanced Graph Algorithms: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.

- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Advanced Graph Algorithms from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Advanced Graph Algorithms with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Advanced Graph Algorithms inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Is Graph Bipartite?	Combination Sum	Median of Two Sorted Arrays
Critical Connections in a Network	Course Schedule	Serialize and Deserialize Binary Tree
Reconstruct Itinerary	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Tarjan

Interview lens: Tarjan questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Tarjan is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Tarjan-style low-link algorithms exist to discover bridges, articulation points, and strongly connected structure in one DFS.

3. Real-World Analogy

Testing which road closures split a city into disconnected neighborhoods.

4. Theory

The theory behind Tarjan is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
tin[u] = entry time
low[u] = earliest reachable ancestor
```

6. Operations

- Assign entry times during DFS.
- Update low values from back edges.
- Identify bridge when low[child] > tin[parent].
- Handle parent edges carefully in undirected graphs.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
import java.util.*;

class Bridges {
    int timer = 0;
    List<int[]> bridges = new ArrayList<>();
    void dfs(int u, int parent, List<List<Integer>> g, int[] tin, int[] low) {
        tin[u] = low[u] = ++timer;
        for (int v : g.get(u)) {
            if (v == parent) continue;
            if (tin[v] == 0) {
                dfs(v, u, g, tin, low);
                low[u] = Math.min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.add(new int[]{u, v});
            } else {
                low[u] = Math.min(low[u], tin[v]);
            }
        }
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Tarjan invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Tarjan before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Tarjan: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions

Level	Representative questions
Beginner	Explain Tarjan from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Tarjan with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Tarjan inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Critical Connections in a Network	Combination Sum	Median of Two Sorted Arrays
Minimum Number of Days to Disconnect Island	Course Schedule	Serialize and Deserialize Binary Tree
Find Critical and Pseudo-Critical Edges in MST	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Kosaraju

Interview lens: Kosaraju questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Kosaraju is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Kosaraju exists to find strongly connected components in directed graphs using two DFS passes.

3. Real-World Analogy

Groups of rooms where every room can reach every other room by one-way corridors.

4. Theory

The theory behind Kosaraju is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
DFS finish order -> reverse graph -> collect SCCs
```

6. Operations

- Run DFS and store finish order.
- Reverse all directed edges.
- Process nodes in reverse finish order.
- Each DFS in reversed graph is one SCC.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
import java.util.*;

class KosarajuSCC {
    static void dfs(int u, List<List<Integer>> g, boolean[] seen, List<Integer> out) {
        seen[u] = true;
        for (int v : g.get(u)) if (!seen[v]) dfs(v, g, seen, out);
        out.add(u);
    }
    static int countScc(List<List<Integer>> g, List<List<Integer>> rg) {
        int n = g.size();
        boolean[] seen = new boolean[n];
        List<Integer> order = new ArrayList<>();
        for (int i = 0; i < n; i++) if (!seen[i]) dfs(i, g, seen, order);
        Arrays.fill(seen, false);
        int scc = 0;
        Collections.reverse(order);
        for (int u : order) if (!seen[u]) {
            dfs(u, rg, seen, new ArrayList<>());
            scc++;
        }
        return scc;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Kosaraju invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Kosaraju before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Kosaraju: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Kosaraju from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Kosaraju with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Kosaraju inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Strongly Connected Components	Combination Sum	Median of Two Sorted Arrays
Minimum Vertices to Reach All Nodes	Course Schedule	Serialize and Deserialize Binary Tree
Longest Cycle in a Graph	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Dynamic Programming

This part groups the dynamic programming topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Dynamic Programming

Interview lens: Dynamic Programming questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Dynamic Programming is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

DP exists when brute force repeats the same subproblems. It stores answers so each state is solved once.

3. Real-World Analogy

A spreadsheet where each cell depends on earlier cells, and recalculation reuses saved values.

4. Theory

The theory behind Dynamic Programming is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
state -> transition -> base case -> answer
```

6. Operations

- Define the state precisely.
- Write recurrence from smaller states.
- Choose memoization or tabulation.
- Optimize space only after correctness is clear.

7. Time Complexity

Operation	Time	Space	Interview note
Memoization	O(states*transition)	O(states)	Top-down
Tabulation	O(states*transition)	O(states)	Bottom-up
Space optimized	Same time	Reduced	Keep needed rows

8. Java Implementation

```
class Knapsack01 {
    static int maxValue(int[] wt, int[] val, int cap) {
        int[] dp = new int[cap + 1];
        for (int i = 0; i < wt.length; i++) {
            for (int c = cap; c >= wt[i]; c--) {
                dp[c] = Math.max(dp[c], val[i] + dp[c - wt[i]]);
            }
        }
        return dp[cap];
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Dynamic Programming invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Dynamic Programming before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Dynamic Programming: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Dynamic Programming from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Dynamic Programming with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Dynamic Programming inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Climbing Stairs	Combination Sum	Median of Two Sorted Arrays
Longest Increasing Subsequence	Course Schedule	Serialize and Deserialize Binary Tree
Edit Distance	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Techniques

This part groups the advanced techniques topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Bit Manipulation

Interview lens: Bit Manipulation questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Bit Manipulation is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Bit manipulation exists because integers are binary sets. It makes parity, masks, subsets, and XOR tricks compact and fast.

3. Real-World Analogy

A row of switches where each switch represents a yes/no property.

4. Theory

The theory behind Bit Manipulation is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
mask = 10110
bit 2 set? mask & (1<<2)
```

6. Operations

- Set, clear, toggle, and test bits.
- Use XOR for cancellation and parity.
- Enumerate subsets with masks.
- Use bitmask DP when n is small but combinations matter.

7. Time Complexity

Operation	Time	Space	Interview note
Get/set/clear bit	O(1)	O(1)	Constant
Count set bits	O(log n)	O(1)	Brian Kernighan
XOR partition	O(n)	O(1)	Pair cancellation

8. Java Implementation

```
class BitTricks {
    static boolean isSet(int x, int bit) { return (x & (1 << bit)) != 0; }
    static int setBit(int x, int bit) { return x | (1 << bit); }
    static int clearBit(int x, int bit) { return x & ~(1 << bit); }
    static int countSetBits(int x) {
        int count = 0;
        while (x != 0) {
            x &= x - 1;
            count++;
        }
        return count;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Bit Manipulation invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Bit Manipulation before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Bit Manipulation: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Bit Manipulation from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Bit Manipulation with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Bit Manipulation inside a disguised story problem, discuss tradeoffs, and produce

Level	Representative questions
	production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Single Number	Combination Sum	Median of Two Sorted Arrays
Counting Bits	Course Schedule	Serialize and Deserialize Binary Tree
Maximum XOR of Two Numbers in an Array	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Backtracking

Interview lens: Backtracking questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Backtracking is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Backtracking exists for controlled exhaustive search where partial decisions can be undone.

3. Real-World Analogy

Trying keys on a lock, removing the wrong key, and trying the next.

4. Theory

The theory behind Backtracking is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
choose -> explore -> undo
```

6. Operations

- Track the current path.
- Mark used choices.
- Prune invalid partial states early.
- Undo every mutation before returning.

7. Time Complexity

Operation	Time	Space	Interview note
Generate subsets	O(2^n)	O(n)	Include/exclude
Permutations	O(n!)	O(n)	Swap/used
Constraint search	Exponential	O(depth)	Prune early

8. Java Implementation

```
import java.util.*;

class Permutations {
    static void permute(int[] a, int idx, List<List<Integer>> ans) {
        if (idx == a.length) {
            List<Integer> one = new ArrayList<>();
            for (int x : a) one.add(x);
            ans.add(one);
            return;
        }
        for (int i = idx; i < a.length; i++) {
            swap(a, idx, i);
            permute(a, idx + 1, ans);
            swap(a, idx, i);
        }
    }
    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Backtracking invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Backtracking before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Backtracking: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Backtracking from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Backtracking with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Backtracking inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Subsets	Combination Sum	Median of Two Sorted Arrays
Combination Sum	Course Schedule	Serialize and Deserialize Binary Tree
N-Queens	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Data Structures

This part groups the advanced data structures topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Segment Tree

Interview lens: Segment Tree questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Segment Tree is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Segment trees exist for fast range queries with updates when prefix sums are not enough.

3. Real-World Analogy

A tournament bracket where each parent summarizes a range of players.

4. Theory

The theory behind Segment Tree is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
[0..7]
 /    \
[0..3] [4..7]
```

6. Operations

- Build aggregate values bottom-up.
- Query by splitting overlapping ranges.
- Update a point or range.
- Use lazy propagation for range updates.

7. Time Complexity

Operation	Time	Space	Interview note
Build	O(n)	O(4n)	Recursive tree
Query/update	O(log n)	O(4n)	Range aggregate
Lazy update	O(log n)	O(4n)	Deferred propagation

8. Java Implementation

```
class SegmentTree {
    int n;
    long[] tree;
    SegmentTree(int[] a) { n = a.length; tree = new long[4 * n]; build(a, 1, 0, n - 1); }
    void build(int[] a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int m = (l + r) >>> 1;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }
    long query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int m = (l + r) >>> 1;
        return query(node * 2, l, m, ql, qr) + query(node * 2 + 1, m + 1, r, ql, qr);
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Segment Tree invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Segment Tree before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Segment Tree: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Segment Tree from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Segment Tree with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Segment Tree inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Range Sum Query - Mutable	Combination Sum	Median of Two Sorted Arrays
Count of Smaller Numbers After Self	Course Schedule	Serialize and Deserialize Binary Tree
My Calendar III	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Fenwick Tree

Interview lens: Fenwick Tree questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Fenwick Tree is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Fenwick trees exist as a compact alternative for prefix queries and point updates.

3. Real-World Analogy

A set of labeled buckets, each bucket stores a power-of-two block summary.

4. Theory

The theory behind Fenwick Tree is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
idx += idx & -idx  // update
idx -= idx & -idx  // query
```

6. Operations

- Add delta to an index.
- Compute prefix sum.
- Convert range sum by subtracting prefixes.
- Use coordinate compression for large values.

7. Time Complexity

Operation	Time	Space	Interview note
Build by updates	O(n log n)	O(n)	Or O(n) variant
Prefix query	O(log n)	O(n)	lowbit
Point update	O(log n)	O(n)	Climb indexes

8. Java Implementation

```
class Fenwick {
    long[] bit;
    Fenwick(int n) { bit = new long[n + 1]; }
    void add(int idx, long delta) {
        for (idx++; idx < bit.length; idx += idx & -idx) bit[idx] += delta;
    }
    long sumPrefix(int idx) {
        long ans = 0;
        for (idx++; idx > 0; idx -= idx & -idx) ans += bit[idx];
        return ans;
    }
    long rangeSum(int l, int r) { return sumPrefix(r) - (l == 0 ? 0 : sumPrefix(l - 1)); }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Fenwick Tree invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Fenwick Tree before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Fenwick Tree: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Fenwick Tree from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Fenwick Tree with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Fenwick Tree inside a disguised story problem, discuss tradeoffs, and produce production-

Level	Representative questions
	clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Range Sum Query - Mutable	Combination Sum	Median of Two Sorted Arrays
Count of Smaller Numbers After Self	Course Schedule	Serialize and Deserialize Binary Tree
Reverse Pairs	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Trees

This part groups the advanced trees topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Binary Lifting

Interview lens: Binary Lifting questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Binary Lifting is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Binary lifting exists to jump through ancestors in powers of two instead of walking one edge at a time.

3. Real-World Analogy

Taking elevator express stops: 1 floor, 2 floors, 4 floors, 8 floors.

4. Theory

The theory behind Binary Lifting is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
up[j][v] = 2^j-th ancestor of v
```

6. Operations

- Precompute jump table.
- Lift by bits of k.
- Equalize depths before LCA.
- Combine with min/max edge tables for path queries.

7. Time Complexity

Operation	Time	Space	Interview note
Preprocess	O(n log n)	O(n log n)	Jump table
kth ancestor	O(log n)	O(1)	Bits of k
LCA	O(log n)	O(1)	Lift depths

8. Java Implementation

```
import java.util.*;

class KthAncestor {
    int LOG;
    int[][] up;
    KthAncestor(int[] parent) {
        int n = parent.length;
        LOG = 1;
        while ((1 << LOG) <= n) LOG++;
        up = new int[LOG][n];
        up[0] = parent.clone();
        for (int j = 1; j < LOG; j++)
            for (int v = 0; v < n; v++)
                up[j][v] = up[j - 1][v] == -1 ? -1 : up[j - 1][up[j - 1][v]];
    }
    int kth(int node, int k) {
        for (int j = 0; j < LOG && node != -1; j++) if (((k >> j) & 1) == 1) node = up[j][node];
        return node;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Binary Lifting invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Binary Lifting before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Binary Lifting: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions

Level	Representative questions
Beginner	Explain Binary Lifting from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Binary Lifting with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Binary Lifting inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Kth Ancestor of a Tree Node	Combination Sum	Median of Two Sorted Arrays
Lowest Common Ancestor of a Binary Tree	Course Schedule	Serialize and Deserialize Binary Tree
Tree Queries	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Euler Tour

Interview lens: Euler Tour questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Euler Tour is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Euler tours flatten trees so subtree questions become range questions.

3. Real-World Analogy

Walking through a museum and timestamping when you enter and leave each room.

4. Theory

The theory behind Euler Tour is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
tin[u] <= tin[v] <= tout[u] means v is in u's subtree
```

6. Operations

- DFS and record entry/exit times.
- Flatten subtree to contiguous range.
- Pair with Fenwick/segment tree for updates.
- Use tin/tout for ancestor checks.

7. Time Complexity

Operation	Time	Space	Interview note
DFS tour	O(n)	O(n)	tin/tout
Subtree query	O(log n)	O(n)	With BIT/segment tree
Ancestor check	O(1)	O(n)	tin/tout containment

8. Java Implementation

```
import java.util.*;

class EulerTour {
    int timer = 0;
    int[] tin, tout;
    void dfs(int u, int parent, List<List<Integer>> tree) {
        tin[u] = timer++;
        for (int v : tree.get(u)) if (v != parent) dfs(v, u, tree);
        tout[u] = timer - 1;
    }
    boolean isAncestor(int u, int v) {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Euler Tour invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Euler Tour before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Euler Tour: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Euler Tour from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Euler Tour with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.

Level	Representative questions
FAANG	Recognize Euler Tour inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Subtree Queries	Combination Sum	Median of Two Sorted Arrays
Employee Importance	Course Schedule	Serialize and Deserialize Binary Tree
Time Needed to Inform All Employees	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Data Structures

This part groups the advanced data structures topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Sparse Table

Interview lens: Sparse Table questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Sparse Table is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Sparse tables exist for immutable range queries where preprocessing can make each query extremely fast.

3. Real-World Analogy

A travel guide that precomputes best hotels for every block length.

4. Theory

The theory behind Sparse Table is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
st[k][i] summarizes range [i, i+2^k-1]
```

6. Operations

- Precompute powers of two.
- Merge two half-blocks.
- Answer idempotent queries with two overlapping blocks.
- Use log table to select block size.

7. Time Complexity

Operation	Time	Space	Interview note
Build	O(n log n)	O(n log n)	Idempotent ops
RMQ query	O(1)	O(1)	Overlapping blocks
Non-idempotent	O(log n)	O(1)	If decomposed

8. Java Implementation

```
class SparseTableMin {
    int[][] st;
    int[] log;
    SparseTableMin(int[] a) {
        int n = a.length;
        log = new int[n + 1];
        for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;
        st = new int[log[n] + 1][n];
        st[0] = a.clone();
        for (int j = 1; j < st.length; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                st[j][i] = Math.min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
    }
    int query(int l, int r) {
        int j = log[r - l + 1];
        return Math.min(st[j][l], st[j][r - (1 << j) + 1]);
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Sparse Table invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Sparse Table before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Sparse Table: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Sparse Table from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine Sparse Table with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Sparse Table inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Range Minimum Query	Combination Sum	Median of Two Sorted Arrays
Sliding Window Maximum	Course Schedule	Serialize and Deserialize Binary Tree
Static Range Queries	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Trees

This part groups the advanced trees topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

LCA

Interview lens: LCA questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

LCA is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Lowest common ancestor exists to answer tree path questions by finding where two root paths meet.

3. Real-World Analogy

Finding the nearest shared manager of two employees.

4. Theory

The theory behind LCA is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
root
 |-- a -- x
 |-- b -- y
LCA(x,y)=root
```

6. Operations

- Preprocess parent/depth.
- Equalize depths.
- Lift both nodes until parents match.
- Use LCA to compute distances and path aggregates.

7. Time Complexity

Operation	Time	Space	Interview note
Core operation	Usually O(log n) to O(n)	Depends	Check constraints
Preprocessing	Often O(n log n)	O(n)	For repeated queries
Query	From O(1) to O(log n)	O(1)	Depends on structure

8. Java Implementation

```
class LCA {
    int LOG, timer = 0;
    int[][] up;
    int[] tin, tout, depth;
    boolean isAncestor(int u, int v) { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
    int lca(int u, int v) {
        if (isAncestor(u, v)) return u;
        if (isAncestor(v, u)) return v;
        for (int j = LOG - 1; j >= 0; j--) {
            int jump = up[j][u];
            if (jump != -1 && !isAncestor(jump, v)) u = jump;
        }
        return up[0][u];
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the LCA invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for LCA before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies LCA: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.
- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain LCA from first principles; implement the core operation; dry-run a small case.

Level	Representative questions
Intermediate	Combine LCA with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize LCA inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Lowest Common Ancestor of a Binary Tree	Combination Sum	Median of Two Sorted Arrays
Lowest Common Ancestor of a BST	Course Schedule	Serialize and Deserialize Binary Tree
Minimum Edge Weight Equilibrium Queries in a Tree	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Advanced Techniques

This part groups the advanced techniques topics from the A2Z preparation path. Read the chapters in order once, then revisit them by pattern when solving mixed problems.

Meet-in-the-Middle

Interview lens: Meet-in-the-Middle questions usually test whether you can identify the invariant before writing code. Start by stating what must remain true after every operation.

1. Concept

Meet-in-the-Middle is a way to organize computation so a repeated decision becomes predictable. In interviews, the concept is less about memorizing a template and more about knowing what information must be preserved while the input changes.

2. Why It Exists

Meet-in-the-middle exists when n is too large for 2^n but small enough for two 2^(n/2) halves.

3. Real-World Analogy

Two search parties start from opposite sides of a mountain and compare notes in the middle.

4. Theory

The theory behind Meet-in-the-Middle is built around an invariant: a statement that stays true as the algorithm progresses. When you can name the invariant, you can prove why the algorithm is correct and explain it under pressure.

A practical interview approach is to write the brute force first in words, identify the repeated work, and then choose the data structure or pattern that removes that repetition.

5. Visual Diagram

```
left subsets + right subsets -> combine by sort/binary search
```

6. Operations

- Split the input into two halves.
- Enumerate all answers in each half.
- Sort one side.
- Use binary search or two pointers to combine.

7. Time Complexity

Operation	Time	Space	Interview note
Split enumerate	O(2^(n/2))	O(2^(n/2))	Two halves
Sort one half	O(m log m)	O(m)	Binary search
Combine	O(m log m)	O(m)	m=2^(n/2)

8. Java Implementation

```
import java.util.*;

class MeetInMiddle {
    static long countSubsetsAtMost(int[] a, int limit) {
        int mid = a.length / 2;
        List<Integer> left = sums(a, 0, mid), right = sums(a, mid, a.length);
        Collections.sort(right);
        long ans = 0;
        for (int x : left) {
            int idx = upperBound(right, limit - x);
            ans += idx;
        }
        return ans;
    }
    static List<Integer> sums(int[] a, int l, int r) {
        List<Integer> res = new ArrayList<>();
        for (int mask = 0; mask < (1 << (r - l)); mask++) {
            int s = 0;
            for (int i = l; i < r; i++) if (((mask >> (i - l)) & 1) == 1) s += a[i];
            res.add(s);
        }
        return res;
    }
    static int upperBound(List<Integer> a, int x) {
        int l = 0, r = a.size();
        while (l < r) { int m = (l + r) >>> 1; if (a.get(m) <= x) l = m + 1; else r = m; }
        return l;
    }
}
```

9. Dry Run

Step 1: Choose a tiny input and write the state before the first operation.

Step 2: Apply the Meet-in-the-Middle invariant after each step, not only at the end.

Step 3: Record the moment the answer changes; this exposes off-by-one mistakes early.

Step 4: Compare the final result with brute force for the same small input.

10. Interview Tricks

Common trap: Do not jump to the template for Meet-in-the-Middle before reading constraints. Constraints decide whether the intended solution is O(n), O(n log n), logarithmic, or exponential with pruning.

- Say the invariant out loud before coding.
- Use names that describe meaning, not just i, j, temp, and flag.
- Dry-run a boundary case: empty input, one element, duplicate values, or disconnected structure.

11. Pattern Recognition

- Look for wording that implies Meet-in-the-Middle: repeated queries, ordered choices, connectivity, contiguous ranges, hierarchy, or state transitions.

- If a brute force solution recomputes the same fact, ask whether preprocessing, a map, a heap, DP, or a tree can store that fact.
- If the input is sorted or can be sorted without breaking meaning, consider binary search, two pointers, or greedy ordering.

12. Common Interview Questions

Level	Representative questions
Beginner	Explain Meet-in-the-Middle from first principles; implement the core operation; dry-run a small case.
Intermediate	Combine Meet-in-the-Middle with sorting, hashing, recursion, or another common pattern.
Advanced	Optimize memory, handle duplicates/edge cases, or prove the invariant under changing constraints.
FAANG	Recognize Meet-in-the-Middle inside a disguised story problem, discuss tradeoffs, and produce production-clean Java.

13. Related LeetCode Problems

Easy	Medium	Hard
Partition Array Into Two Arrays to Minimize Sum Difference	Combination Sum	Median of Two Sorted Arrays
Closest Subsequence Sum	Course Schedule	Serialize and Deserialize Binary Tree
Split Array With Same Average	Longest Substring Without Repeating Characters	Minimum Window Substring

14. Practice Roadmap

Stage	Focus	Exit criteria
Easy	Understand the operation and write the simplest correct version.	You can solve without looking at notes.
Medium	Add constraints, edge cases, and pattern combinations.	You can explain why the complexity is enough.
Hard	Optimize, prove, and handle adversarial cases.	You can compare at least two approaches clearly.

Binary Search Pattern Handbook

Binary Search Pattern Handbook is a decision guide. Use it before coding to choose the smallest sufficient template and avoid over-engineering.

Pattern	Recognize it by	Implementation focus

Pattern	Recognize it by	Implementation focus
Exact search	Find target in sorted array	Maintain inclusive or half-open bounds consistently.
Lower/upper bound	First index satisfying condition	Use lo < hi and return lo.
Search on answer	Minimum feasible capacity/time/radius	Write a monotonic check first.
Rotated arrays	One half is sorted	Decide which half can contain target.
2D matrix	Flattened sorted or staircase property	Respect row/column monotonicity.

Interview note: A pattern name is not a proof. Always pair it with an invariant and a dry run.

Sliding Window Pattern Handbook

Sliding Window Pattern Handbook is a decision guide. Use it before coding to choose the smallest sufficient template and avoid over-engineering.

Pattern	Recognize it by	Implementation focus
Fixed window	Exactly k length	Initialize first k, then move one step at a time.
Variable window	At most condition	Expand right, shrink left while invalid.
At most K	Count subarrays/substrings	Every valid window contributes right-left+1.
Exactly K	Exact count	Compute atMost(K)-atMost(K-1).
Min window	Cover required characters	Shrink after all requirements are satisfied.

Interview note: A pattern name is not a proof. Always pair it with an invariant and a dry run.

Graph Pattern Handbook

Graph Pattern Handbook is a decision guide. Use it before coding to choose the smallest sufficient template and avoid over-engineering.

Pattern	Recognize it by	Implementation focus
DFS	Components, recursion, cycle state	Track parent for undirected graphs.
BFS	Shortest unweighted paths	Process by layers.
Grid	Cells as nodes	Use direction arrays and bounds checks.
Topological	Prerequisites	DAG only; cycle means impossible.
Shortest path	Weighted navigation	Choose BFS, Dijkstra, or Bellman-Ford by weights.
MST	Connect all with minimum cost	Kruskal uses DSU; Prim uses heap.
DSU	Dynamic connectivity	Path compression plus union by size.
Bipartite	Two-color graph	Odd cycle breaks it.

Interview note: A pattern name is not a proof. Always pair it with an invariant and a dry run.

DP Pattern Handbook

DP Pattern Handbook is a decision guide. Use it before coding to choose the smallest sufficient template and avoid over-engineering.

Pattern	Recognize it by	Implementation focus
0/1 Knapsack	Take or skip each item once	Iterate capacity backward for 1D optimization.
Unbounded	Reuse items	Iterate capacity forward.
LIS	Increasing sequence	O(n log n) tails array or O(n^2) DP.
LCS	Two strings	State i,j over prefixes.
Grid	Move right/down or four directions	Base row/column carefully.
Partition	Split array/string	Try last cut or subset sum.
Digit DP	Count numbers with constraints	State pos,tight,started,...
Bitmask DP	Small n assignment/TSP	State mask and last/position.
Tree DP	Parent-child decisions	Return include/exclude or multiple states.
Interval DP	Merge/partition intervals	Process by length.
State compression	Reduce memory	Only keep states needed for next transition.

Interview note: A pattern name is not a proof. Always pair it with an invariant and a dry run.

Backtracking Pattern Handbook

Backtracking Pattern Handbook is a decision guide. Use it before coding to choose the smallest sufficient template and avoid over-engineering.

Pattern	Recognize it by	Implementation focus
Subsets	Choose/not choose	No need to use every element.
Permutations	Order matters	Track used elements or swap in place.
Combination Sum	Reuse or no reuse	Control start index.
N Queens	Constraint placement	Track columns and diagonals.
Sudoku	Fill constrained cells	Choose next empty cell with fewest candidates.
Word Search	Path in grid	Mark visited, explore, then unmark.

Interview note: A pattern name is not a proof. Always pair it with an invariant and a dry run.

FAANG Interview Strategy

Company style	What usually matters	Preparation emphasis
Google	Problem solving clarity, edge cases, generalization	Practice explaining invariants before code.
Meta	Speed, clean implementation, follow-up optimization	Drill common patterns until templates are automatic.
Microsoft	Balanced coding, communication, practical debugging	Talk through tradeoffs and test cases steadily.
Amazon	Correctness, ownership stories, scalable reasoning	Pair DSA practice with leadership-principle examples.
Apple/Nvidia/Adobe/ Atlassian/Uber	Role-dependent depth and robust engineering taste	Expect DSA plus system/product constraints.

- Start every answer with brute force, then optimize.
- Write tests before declaring done: empty, one element, duplicates, extremes, and impossible cases.
- Narrate tradeoffs without rambling: time, space, correctness, and implementation risk.

OA Strategy

Phase	Time box	Action
Scan	3-5 minutes	Read all problems and identify obvious patterns.
First solve	20-25 minutes	Do the highest-confidence problem first.
Core attempt	35-45 minutes	Implement the main problem with careful tests.
Fallback	Last 10 minutes	Submit partial brute force only if constraints allow partial scoring.

Debugging strategy: Print only small state locally. On platform submissions, reason from failed edge cases instead of flooding output.

Revision Checklist

Topic group	Must be able to do
Foundations	Explain Big-O, recursion base cases, hashing collisions, and Java collection costs.
Arrays and strings	Use prefix sums, two pointers, sliding windows, Kadane, and binary search.
Linked structures	Reverse lists, detect cycles, use stack/queue/deque, and handle null boundaries.
Trees	Traverse, compute height/diameter/views, validate BST, and answer LCA-style questions.
Graphs	Run BFS/DFS, topological sort, Dijkstra, MST, DSU, SCC, bridges, and grid traversal.
DP	Define states for knapsack, LIS, LCS, grid, partition, interval, digit, tree, and bitmask DP.
Advanced DS	Implement Fenwick, segment tree, sparse table, binary lifting, Euler tour, and trie.

30-Day Revision Plan

Time	Focus
Days 1-5	Foundations, arrays, strings, hashing, sorting
Days 6-10	Binary search, two pointers, sliding window, prefix/difference/Kadane
Days 11-15	Linked list, stack, queue, heap, trie
Days 16-21	Trees, BST, recursion, backtracking
Days 22-27	Graphs, DSU, shortest path, MST, topo, SCC
Days 28-30	DP patterns, mock interviews, final weak spots

Rule: Every day should include at least one dry run by hand and one reflection note on a mistake.

60-Day Preparation Plan

Time	Focus
Weeks 1-2	Complete foundations and basic arrays/strings with daily dry runs.
Weeks 3-4	Master binary search, sliding window, two pointers, linked list, stack, queue.
Weeks 5-6	Trees, BST, heap, trie, backtracking, and greedy.
Weeks 7-8	Graphs and DP with timed practice.
Final days	Mock interviews and revision checklist.

Rule: Every day should include at least one dry run by hand and one reflection note on a mistake.

90-Day FAANG Preparation Roadmap

Time	Focus
Month 1	Build foundations and solve breadth-first across all easy/medium topics.
Month 2	Deepen graphs, DP, binary search on answer, and advanced data structures.
Month 3	Timed mocks, hard problems, company-specific practice, and explanation polish.

Rule: Every day should include at least one dry run by hand and one reflection note on a mistake.

Final 7-Day Interview Revision

Time	Focus
Day 1	Complexity, Java collections, arrays, strings.
Day 2	Binary search, sliding window, two pointers.
Day 3	Linked list, stack, queue, heap, trie.
Day 4	Trees, BST, LCA, binary lifting.
Day 5	Graphs, shortest path, MST, DSU, SCC.
Day 6	DP and backtracking.
Day 7	Mock interview, templates, rest, and logistics.

Rule: Every day should include at least one dry run by hand and one reflection note on a mistake.

Appendix: Big-O Cheat Sheet

Complexity Analysis Cheat Sheet

Notation	Meaning	Interview wording
Big-O	Upper bound on growth	The algorithm will not grow worse than this class.
Big-Theta	Tight bound	The algorithm grows exactly in this class asymptotically.
Big-Omega	Lower bound	The algorithm must take at least this much in the best guaranteed sense.
Amortized	Average over a sequence	Some operations are expensive, but the total sequence is controlled.
Recursive	Cost defined by recurrence	Count branches, work per level, and depth.
Master theorem	Shortcut for T(n)=aT(n/b)+f(n)	Compare f(n) with n^(log_b a).

- O(1): direct access or constant fixed work.
- O(log n): the search space shrinks by a factor each step.
- O(n): every element is processed a constant number of times.
- O(n log n): sorting or balanced divide-and-conquer.
- O(n^2): pair comparisons; usually too slow beyond about 10^5.
- O(2^n) and O(n!): subset/permutation search; require small n or pruning.

Warning: Never say a recursive algorithm is O(n) just because it has one parameter. Draw the recursion tree or define states.

Sorting Comparison Table

Algorithm	Best	Average	Worst	Stable	Use
Bubble	O(n)	O(n^2)	O(n^2)	Yes	Learning only
Insertion	O(n)	O(n^2)	O(n^2)	Yes	Nearly sorted small arrays
Merge	O(n log n)	O(n log n)	O(n log n)	Yes	Stable sorting and inversion count
Quick	O(n log n)	O(n log n)	O(n^2)	No	In-place average speed
Heap	O(n log n)	O(n log n)	O(n log n)	No	Memory-sensitive sorting

Tree Traversal Table

Traversal	Order	Use
Preorder	Root, left, right	Copy/serialize structure
Inorder	Left, root, right	BST sorted order
Postorder	Left, right, root	Delete/free or compute child facts first
Level order	Breadth first	Shortest layers and views

Graph Algorithm Comparison

Algorithm	Graph type	Purpose
BFS	Unweighted	Shortest edge count
Dijkstra	Nonnegative weighted	Shortest weighted path
Bellman-Ford	Negative edges	Shortest path and negative-cycle detection
Kruskal/Prim	Undirected weighted	Minimum spanning tree
Kosaraju/Tarjan	Directed	Strongly connected components

DP Pattern Comparison

Pattern	State	Typical transition
Knapsack	i, capacity	take or skip
Grid	row, col	from top/left or neighbors
LCS	i, j	match or drop one side
LIS	i or tails length	extend smaller tail
Interval	l, r	try split point

Java Collections Comparison

Type	Ordered?	Duplicates?	Primary method
HashSet	No	No	contains/add/remove
TreeSet	Sorted	No	floor/ceiling
HashMap	No	Keys unique	get/put
TreeMap	Sorted keys	Keys unique	floorKey/ceilingKey
PriorityQueue	Heap order	Yes	offer/poll/peek

Common Formula Sheet

Formula	Use
sum 1..n = n(n+1)/2	Counting pairs, arithmetic ranges
subarrays in length n = n(n+1)/2	Counting contiguous choices
subsets = 2^n	Bitmask/backtracking
permutations = n!	Ordering all elements
edges in complete graph = n(n-1)/2	Dense graph bounds

Bit Manipulation Tricks

Expression	Meaning
x & -x	Lowest set bit
x & (x - 1)	Remove lowest set bit
x ^ x = 0	Pair cancellation
mask | (1 << b)	Set bit b
mask & ~(1 << b)	Clear bit b

Recursion Template

```
class RecursionTemplate {
    static void subsets(int[] a, int idx, java.util.List<Integer> path,
                        java.util.List<java.util.List<Integer>> ans) {
        if (idx == a.length) {
            ans.add(new java.util.ArrayList<>(path));
            return;
        }
        subsets(a, idx + 1, path, ans);       // do not take
        path.add(a[idx]);
        subsets(a, idx + 1, path, ans);       // take
        path.remove(path.size() - 1);         // undo
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

DFS Template

```
import java.util.*;

class GraphTraversal {
    static List<Integer> bfs(List<List<Integer>> g, int start) {
        boolean[] seen = new boolean[g.size()];
        List<Integer> order = new ArrayList<>();
        Queue<Integer> q = new ArrayDeque<>();
        seen[start] = true;
        q.offer(start);
        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g.get(u)) if (!seen[v]) {
                seen[v] = true;
                q.offer(v);
            }
        }
        return order;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

BFS Template

```
import java.util.*;

class GraphTraversal {
    static List<Integer> bfs(List<List<Integer>> g, int start) {
        boolean[] seen = new boolean[g.size()];
        List<Integer> order = new ArrayList<>();
        Queue<Integer> q = new ArrayDeque<>();
        seen[start] = true;
        q.offer(start);
        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g.get(u)) if (!seen[v]) {
                seen[v] = true;
                q.offer(v);
            }
        }
        return order;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Binary Search Template

```
class BinarySearchPatterns {
    static int lowerBound(int[] a, int target) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] >= target) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Union Find Template

```
class UnionFind {
    int[] parent, size;
    UnionFind(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) { int t = a; a = b; b = t; }
        parent[b] = a; size[a] += size[b];
        return true;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Segment Tree Template

```
class SegmentTree {
    int n;
    long[] tree;
    SegmentTree(int[] a) { n = a.length; tree = new long[4 * n]; build(a, 1, 0, n - 1); }
    void build(int[] a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int m = (l + r) >>> 1;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }
    long query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int m = (l + r) >>> 1;
        return query(node * 2, l, m, ql, qr) + query(node * 2 + 1, m + 1, r, ql, qr);
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Trie Template

```
class Trie {
    static class Node {
        Node[] next = new Node[26];
        boolean end;
    }
    private final Node root = new Node();
    void insert(String word) {
        Node cur = root;
        for (char ch : word.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new Node();
            cur = cur.next[i];
        }
        cur.end = true;
    }
    boolean startsWith(String prefix) {
        Node cur = root;
        for (char ch : prefix.toCharArray()) {
            cur = cur.next[ch - 'a'];
            if (cur == null) return false;
        }
        return true;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Monotonic Stack Template

```
import java.util.*;

class NextGreaterElement {
    static int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        Deque<Integer> st = new ArrayDeque<>(); // indexes with decreasing values
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && nums[i] > nums[st.peek()]) ans[st.pop()] = nums[i];
            st.push(i);
        }
        return ans;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Sliding Window Template

```
import java.util.*;

class LongestAtMostKDistinct {
    static int longest(String s, int k) {
        Map<Character, Integer> count = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            count.put(s.charAt(right), count.getOrDefault(s.charAt(right), 0) + 1);
            while (count.size() > k) {
                char c = s.charAt(left++);
                count.put(c, count.get(c) - 1);
                if (count.get(c) == 0) count.remove(c);
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Two Pointer Template

```
class TwoPointerPairSum {
    static boolean hasPair(int[] sorted, int target) {
        int l = 0, r = sorted.length - 1;
        while (l < r) {
            int sum = sorted[l] + sorted[r];
            if (sum == target) return true;
            if (sum < target) l++;
            else r--;
        }
        return false;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Fast Input Template

```
import java.io.*;
import java.util.*;

public class Main {
    static class FastScanner {
        private final InputStream in = System.in;
        private final byte[] buffer = new byte[1 << 16];
        private int ptr = 0, len = 0;
        int read() throws IOException {
            if (ptr >= len) { len = in.read(buffer); ptr = 0; }
            return len <= 0 ? -1 : buffer[ptr++];
        }
        int nextInt() throws IOException {
            int c, sign = 1, val = 0;
            do { c = read(); } while (c <= ' ' && c != -1);
            if (c == '-') { sign = -1; c = read(); }
            while (c > ' ') { val = val * 10 + c - '0'; c = read(); }
            return val * sign;
        }
    }
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner();
        int n = fs.nextInt();
        long sum = 0;
        for (int i = 0; i < n; i++) sum += fs.nextInt();
        System.out.println(sum);
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.

Competitive Programming Utilities

```
class MathToolkit {
    static long gcd(long a, long b) {
        while (b != 0) {
            long t = a % b;
            a = b;
            b = t;
        }
        return Math.abs(a);
    }

    static long modPow(long a, long e, long mod) {
        long ans = 1 % mod;
        while (e > 0) {
            if ((e & 1) == 1) ans = (ans * a) % mod;
            a = (a * a) % mod;
            e >>= 1;
        }
        return ans;
    }
}
```

Usage: Rewrite this template from memory at least twice. In interviews, templates are only useful when you understand the invariant behind each line.